

IZMIR INSTITUTE OF TECHNOLOGY

Department of Computer Engineering

CENG463 - Introduction to Machine Learning

Take-Home Midterm Examination Question 2

Name: Behice Kadioğlu

Student ID: 300201123

Instructor: Prof. Dr. Aytuğ Onan

Date 4 May 2026

Contents

1	Introduction	3
1.1	Problem Formulation	3
1.2	Dataset Description	3
2	Exploratory Data Analysis (EDA)	3
2.1	Class Distribution and Imbalance Ratio	4
2.2	Feature Analysis and Preprocessing	4
2.3	Correlation and Feature Interactions	4
3	Methodology	5
3.1	Data Preprocessing and Leakage Prevention	5
3.2	Resampling Strategies	6
3.3	Model Architectures and Cost-Sensitive Learning	6
3.4	Probability Calibration	6
4	Experimental Setup	6
4.1	Computational Environment	7
4.2	Reproducibility and Hyperparameters	7
4.3	Validation Strategy	7
5	Evaluation and Results	7
5.1	Performance Metrics Comparison	7
5.2	Precision-Recall and ROC Analysis	8
5.3	Probability Calibration Results	9
6	Discussion	10
6.1	Threshold Optimization and Precision-Recall Trade-off	10
6.2	Cost Analysis: False Negatives vs. False Positives	10
6.3	Comparison of Strategies	11
7	Conclusion	12

1 Introduction

In the field of predictive modeling, class imbalance occurs when the frequency of one class significantly outweighs the others. In the context of this project, we address the challenge of "Extreme Imbalance," where the target class represents a minute fraction of the total observations. This study evaluates the effectiveness of various algorithmic and data-centric strategies—including resampling, cost-sensitive learning, and probability calibration—to build a robust fraud detection system.

1.1 Problem Formulation

The primary objective of this project is to develop a classifier capable of identifying rare fraudulent transactions within a vast stream of legitimate ones. Standard machine learning algorithms are typically designed to maximize overall accuracy, which often leads to a "majority class bias" where the model ignores the minority class entirely while still achieving high accuracy scores.

In a real-world financial environment, the "cost" of a False Negative (missing a fraudulent transaction) is significantly higher than that of a False Positive (flagging a legitimate transaction for review). Therefore, this study seeks to optimize models based on specialized metrics such as Precision-Recall AUC (PR-AUC) and the Matthews Correlation Coefficient (MCC), rather than standard accuracy.

1.2 Dataset Description

For this analysis, we utilized the Credit Card Fraud Detection dataset. The dataset comprises transactions made by credit cards in September 2013 by European cardholders. The characteristics of the data are as follows:

- **Total Samples:** 284,807
- **Minority Class (Fraud) Samples:** 492
- **Majority Class (Legitimate) Samples:** 284,315
- **Imbalance Ratio (IR):** 577.9:1

The dataset includes 28 numerical features (*V1* through *V28*) resulting from a PCA transformation, along with *Amount* and *Time*.

To ensure a valid experimental setup, the data was preprocessed by log-transforming the skewed *Amount* feature and dropping the *Time* feature to prevent data leakage.

2 Exploratory Data Analysis (EDA)

The objective of the EDA phase was to understand the underlying structure of the feature space and the severity of the class overlap in the presence of extreme imbalance.

2.1 Class Distribution and Imbalance Ratio

The primary characteristic of this dataset is its extreme skewness. Out of 284,807 transactions, only 492 are labeled as fraud, while 284,315 are legitimate. We calculated the IR to be 577.9:1, meaning that for every single fraudulent transaction, there are approximately 578 legitimate ones.

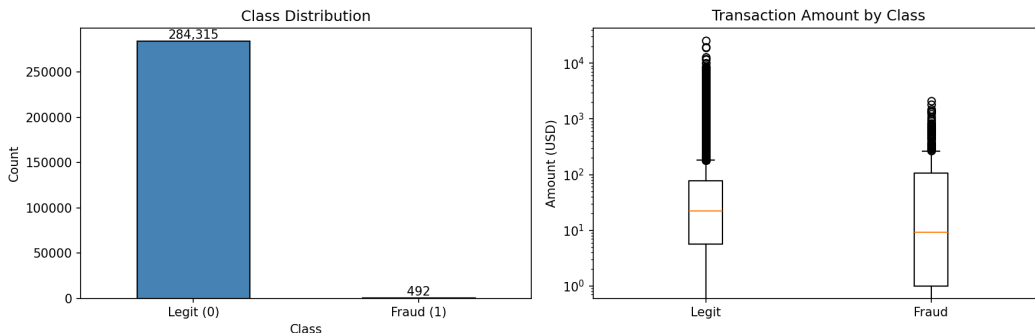


Figure 1: Class distribution (showing extreme imbalance) and log-transformed transaction amounts by class.

As shown in Figure 1 (left), the fraud class is nearly invisible on a linear scale, emphasizing the need for specialized evaluation metrics like PR-AUC over standard accuracy.

2.2 Feature Analysis and Preprocessing

The dataset contains 30 features, including 28 PCA-transformed variables ($V1-V28$), **Time**, and **Amount**.

The **Amount** feature exhibited a very wide range and heavy skewness. To compress this range and make the feature more suitable for gradient-based models like MLP and XGBoost, a log-transformation ($\log_{10}$) was applied. Figure 1 (right) illustrates the distribution of transaction amounts after this transformation across both classes.

We identified that the **Time** feature represents the seconds elapsed between each transaction and the first transaction in the dataset. Since this is a relative measure rather than an absolute timestamp, it could introduce data leakage by providing a proxy for the sequence of transactions; thus, it was dropped from the feature matrix.

2.3 Correlation and Feature Interactions

To identify the most predictive variables, we performed a correlation analysis.

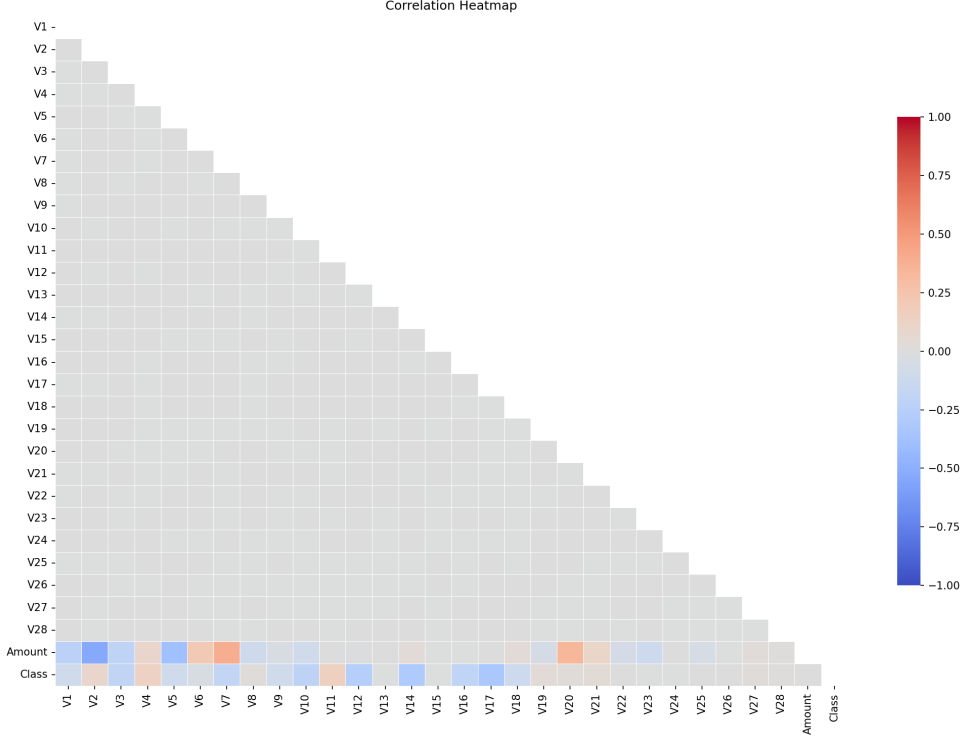


Figure 2: Correlation heatmap for the numerical features and target variable.

Figure 2 displays the interaction between the numerical features. Most PCA-transformed features (V1-V28) show little to no correlation with each other, which is expected from a principal component analysis. However, specific features like V17, V14, and V12 showed notable inverse correlations with the target class, suggesting they are critical indicators for identifying fraudulent activity.

3 Methodology

To address the extreme imbalance of the Credit Card Fraud dataset, a multi-faceted methodological approach was adopted, combining robust preprocessing, data-level resampling, and algorithmic-level cost adjustments.

3.1 Data Preprocessing and Leakage Prevention

A critical component of the experimental setup was the prevention of data leakage. The **Time** feature was removed to ensure the model did not learn relative temporal patterns that might not generalize. The **Amount** feature underwent a log-transformation, $Amount_{trans} = \ln(1 + Amount)$, to stabilize variance and minimize the impact of extreme outliers. All numerical features were normalized using **StandardScaler**. To ensure that scaling and resampling parameters were derived strictly from the training folds, we utilized the **ImbPipeline**. This ensures that the test set remains completely "unseen," which is vital for achieving the reproducible results required by the project.

3.2 Resampling Strategies

We implemented three resampling strategies to evaluate their impact on the minority class recall:

- SMOTE (Synthetic Minority Over-sampling Technique): Generates synthetic examples by interpolating between existing minority samples.
- ADASYN (Adaptive Synthetic): Focuses on generating data in "hard to learn" regions where minority class density is low.
- Random Undersampling: Reduces the majority class to match the minority count, providing a baseline for data-level balancing.

3.3 Model Architectures and Cost-Sensitive Learning

Four distinct classifiers were implemented to provide a comprehensive comparison:

- Logistic Regression: Employed as a baseline linear model with $L2$ regularization.
- Random Forest: A robust ensemble method tested with both balanced class weights and resampling strategies.
- XGBoost (Extreme Gradient Boosting): Configured with the `scale_pos_weight` parameter set to the imbalance ratio (577.9), which scales the gradient for the positive class.
- Neural Network (MLP): A deep architecture with three hidden layers (128, 64, and 32 neurons) using ReLU activations and Keras-based class weights to penalize minority class errors.

3.4 Probability Calibration

Since many classifiers (especially tree-based ensembles and neural networks) produce distorted probability estimates under extreme imbalance, we applied probability calibration.

Platt Scaling (Sigmoid) transforms model outputs into calibrated probabilities using a logistic regression wrapper.

Isotonic Regression is a non-parametric approach that fits a non-decreasing function to the predicted probabilities, which is particularly effective for large datasets.

4 Experimental Setup

This section details the computational environment, the configurations for reproducibility, and the rigorous validation strategy employed to ensure the reliability of the findings.

4.1 Computational Environment

The experiments were conducted in a Google Colab environment utilizing an A100 GPU for accelerated training of the Neural Network and XGBoost models. The primary libraries used for implementation include:

- Scikit-learn: For core machine learning algorithms and preprocessing.
- XGBoost: For the gradient boosting implementation.
- TensorFlow/Keras (via SciKeras): For the Multi-Layer Perceptron (MLP) architecture.
- Imbalanced-learn (imblearn): For resampling pipelines and specialized balancing techniques.

4.2 Reproducibility and Hyperparameters

In accordance with the project requirements, all experiments were governed by a fixed random seed of 22.

All numerical features were standardized using `StandardScaler` within the cross-validation pipeline to prevent feature magnitude bias.

Model-Specific Parameters:

- Random Forest: 200 estimators with `n_jobs=-1`.
- XGBoost: 300 estimators, `scale_pos_weight = 577.9`, and `eval_metric="logloss"`.
- MLP: 20 epochs with a batch size of 32.

4.3 Validation Strategy

To provide an unbiased evaluation of the models on the extremely imbalanced dataset, we utilized a Stratified 5-Fold Cross-Validation strategy.

Stratification ensures that the extreme imbalance ratio (IR) of 577.9:1 is preserved across each training and validation fold, preventing folds with zero minority samples.

We collected Out-of-Fold (OOF) probabilities and predictions for the entire dataset. These OOF results were then used to generate the final performance metrics, PR-Curves, and Calibration plots, ensuring that no model was evaluated on data it had seen during training.

5 Evaluation and Results

5.1 Performance Metrics Comparison

The results for all implemented classifiers and resampling strategies are summarized in Table 1. The metrics reveal a clear distinction between the performance of ensemble-based models and the baseline/neural network approaches.

Table 1: Comprehensive Performance Metrics for All Models (OOF)

Model	PR-AUC	MCC	F1 (Macro)	Recall	Precision	Balanced Acc.
LogReg	0.7593	0.7392	0.8641	0.6220	0.8793	0.8109
RF	0.8470	0.8650	0.9308	0.7927	0.9443	0.8963
XGBoost (SPW)	0.8587	0.8666	0.9325	0.8150	0.9218	0.9075
MLP (Keras CW)	0.0845	0.2565	0.5654	0.8841	0.0762	0.9328
RF + SMOTE	0.8481	0.8503	0.9248	0.8171	0.8855	0.9084
RF + ADASYN	0.8339	0.8399	0.9195	0.8008	0.8814	0.9003
RF + RandUnder	0.7328	0.2103	0.5403	0.9085	0.0505	0.9395
Random Forest (Weighted)	0.8480	0.8420	0.9181	0.7480	0.9485	0.8739

XGBoost (SPW) achieved the highest PR-AUC (0.8587) and MCC (0.8666), suggesting it is the most effective model for discriminating fraud in this highly imbalanced environment. In contrast, while RF + Random Undersampling and MLP achieved high recall (0.9085 and 0.8841 respectively), their exceptionally low precision resulted in very poor PR-AUC and MCC scores, indicating a high volume of false alarms.

5.2 Precision-Recall and ROC Analysis

The visual comparison of the threshold-invariant curves is provided in Figure 3 and 4.

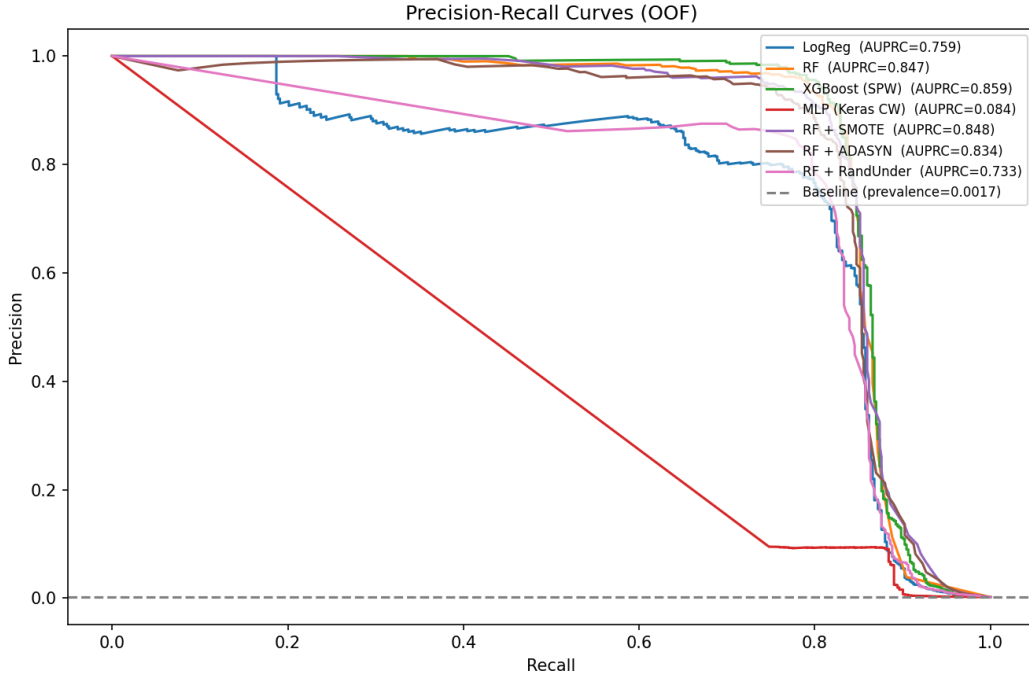


Figure 3: Precision-Recall curves for all OOF predictions. The horizontal dashed line represents the baseline prevalence of fraud (0.0017).

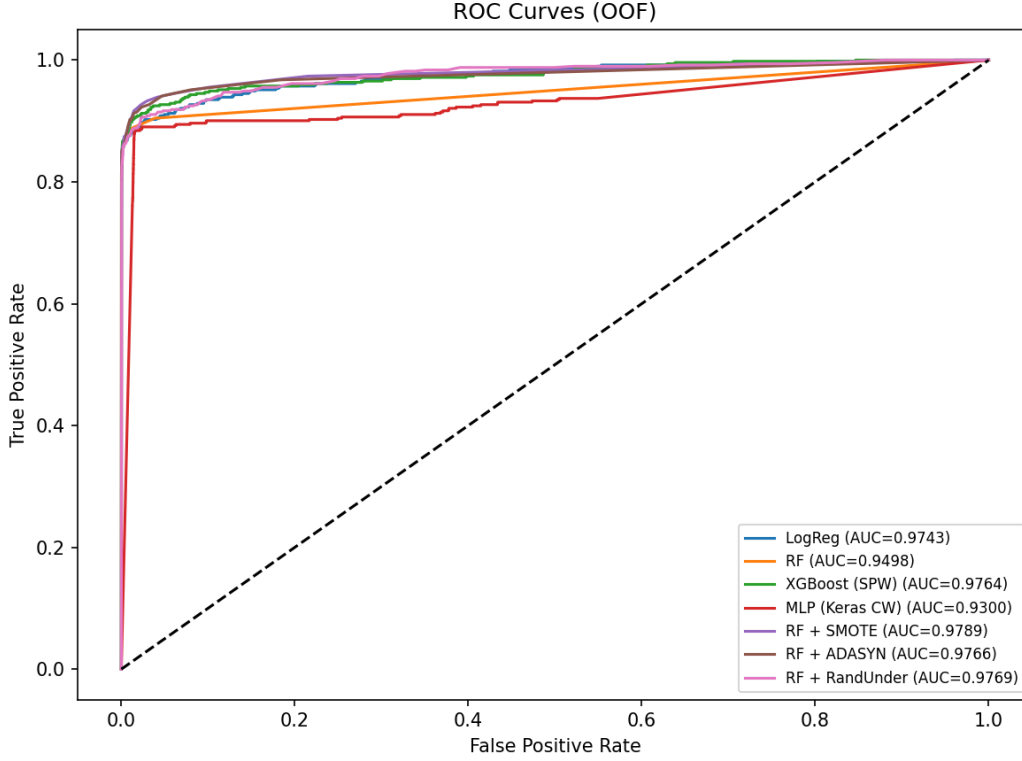


Figure 4: ROC curves for OOF predictions across all models.

The Precision-Recall curves (Figure 3) highlight the superiority of XGBoost and Random Forest over the baseline. As the recall increases, XGBoost maintains a higher precision compared to other models, which is critical for minimizing the operational cost of fraud investigations.

5.3 Probability Calibration Results

Probability calibration was performed on the two best-performing models by PR-AUC: XGBoost (SPW) and RF + SMOTE. We compared the raw outputs against Platt Scaling and Isotonic Regression.

Table 2: Brier Score Comparison (Calibration)			
Model	Raw	Platt (Sigmoid)	Isotonic
XGBoost (SPW)	0.00041	0.00041	0.00040
RF + SMOTE	0.00055	0.00045	0.00044

The Reliability Diagrams in Figure 5 demonstrate that both models were initially poorly calibrated. Isotonic Regression provided the most significant improvement, yielding the lowest Brier scores for both models and bringing the predicted probabilities closer to the identity line.

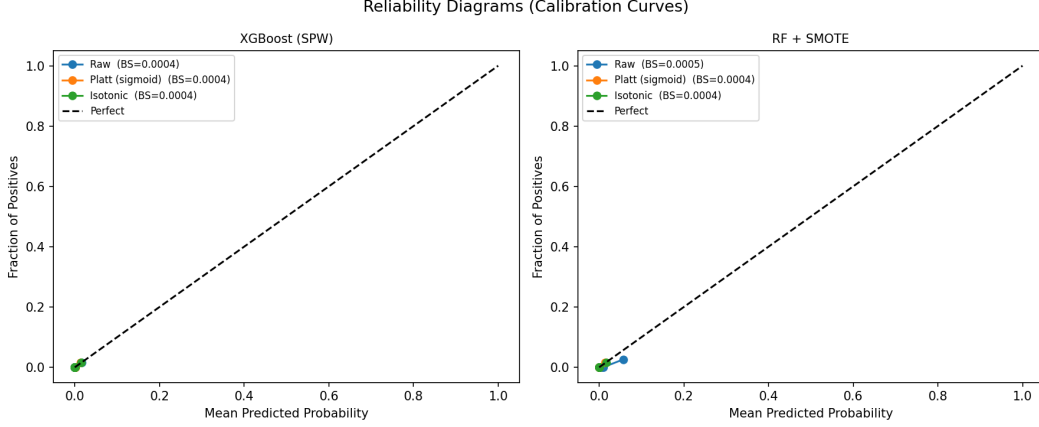


Figure 5: Reliability Diagrams (Calibration Curves) comparing raw and calibrated probabilities.

6 Discussion

6.1 Threshold Optimization and Precision-Recall Trade-off

In fraud detection, the default classification threshold of 0.5 is rarely optimal because the cost of misclassifying a minority sample (fraud) is disproportionately high. We performed threshold optimization by maximizing the F1-score to find the most balanced operating point for each model.

Table 3: Optimal Threshold Analysis for Best Performing Models

Model	Opt. Threshold	F1 @ Threshold	Precision	Recall
LogReg	0.0744	0.7860	0.7736	0.7988
RF	0.5100	0.8647	0.9512	0.7927
XGBoost (SPW)	0.9036	0.8722	0.9519	0.8049
RF + SMOTE	0.5500	0.8556	0.9029	0.8130

The results in Table 3 show that XGBoost (SPW) reached its peak performance at a high threshold of 0.9036. This indicates that the model’s raw probability estimates were pushed toward the extremes by the heavy `scale_pos_weight` (577.9), requiring a stringent threshold to maintain high precision (0.9519) while still capturing over 80% of fraudulent transactions.

6.2 Cost Analysis: False Negatives vs. False Positives

The real-world utility of a fraud detection system is determined by its ability to minimize financial loss (False Negatives) without excessive administrative overhead (False Positives).

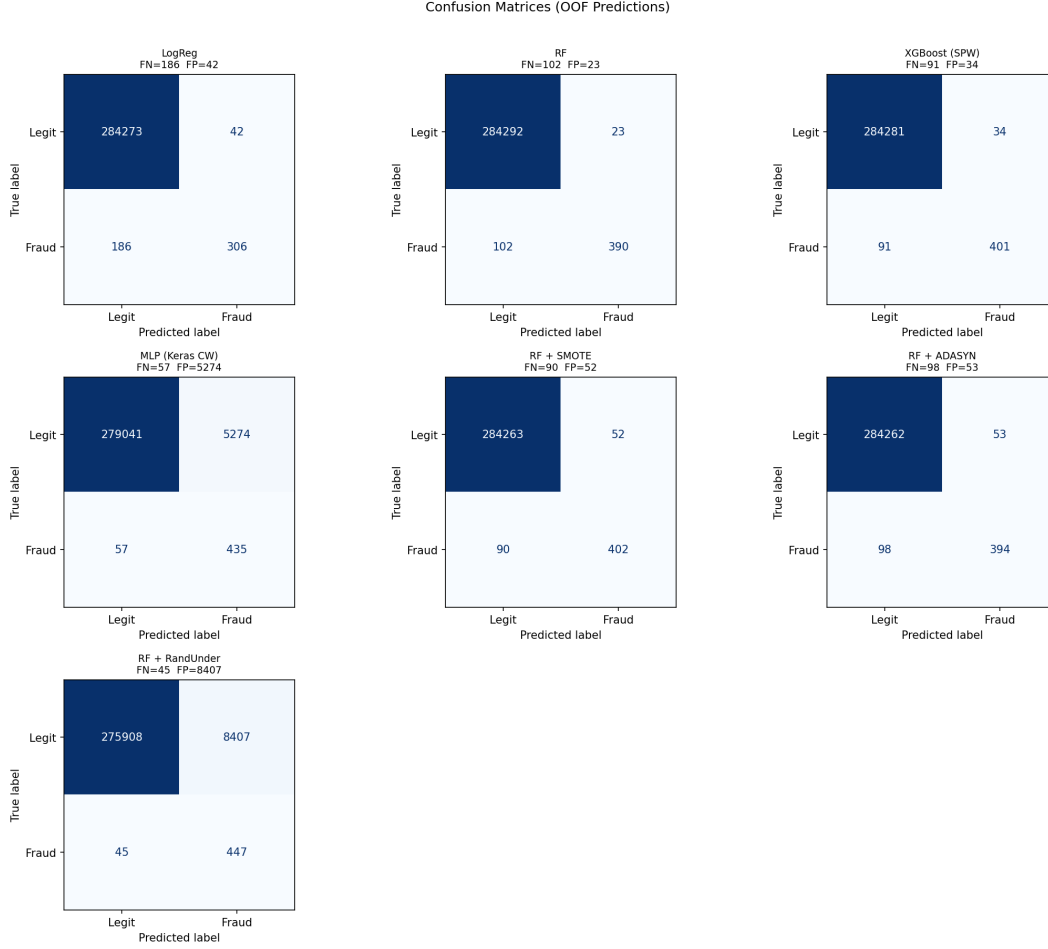


Figure 6: Confusion Matrices for OOF Predictions

As visualized in the confusion matrices (Figure 6), XGBoost (SPW) identified 401 out of 492 frauds, resulting in 91 False Negatives. Crucially, it only produced 34 False Positives. Every False Negative represents a successful theft. Missing 91 transactions is a significant improvement over the baseline LogReg, which missed 186. With only 34 False Positives across nearly 285,000 transactions, the operational cost of manually verifying these alerts is extremely low. This precision ensures customer satisfaction by reducing the number of legitimate transactions that are incorrectly blocked.

6.3 Comparison of Strategies

A key finding in this study is the competitiveness of Cost-Sensitive Learning (via `scale_pos_weight` or class weights) compared to Resampling methods. While RF + SMOTE performed admirably, achieving a PR-AUC of 0.8481, it did not significantly outperform the purely algorithmic weighting of XGBoost (0.8587). Furthermore, resampling can be computationally expensive on large datasets, whereas cost-sensitive parameters provide a more efficient mechanism for handling imbalance without the need to generate synthetic data points.

7 Conclusion

The identification of fraudulent credit card transactions within a dataset characterized by an extreme imbalance ratio of 577.9 : 1 requires a departure from standard machine learning practices. This study has demonstrated that while several strategies can improve minority class recognition, algorithmic-level cost-sensitive learning—specifically through the use of XGBoost with the `scale_pos_weight` parameter—provides the most robust performance for this task.

The key takeaways from this investigation are as follows:

- Traditional accuracy and ROC-AUC proved to be misleadingly optimistic due to the overwhelming majority of legitimate transactions. The use of PR-AUC and MCC was essential to accurately distinguish high-performing models like XGBoost ($PR-AUC = 0.8587$) from models like the MLP, which suffered from extremely low precision.
- While data-level resampling methods such as SMOTE and ADASYN showed significant improvements over the baseline, they did not surpass the purely cost-sensitive weighting employed by XGBoost. This suggests that for large datasets, tuning algorithmic parameters to penalize minority class errors is both more efficient and effective than synthetic data generation.
- Reliability diagrams revealed that high-performing models are often poorly calibrated under extreme imbalance. The application of Isotonic Regression successfully aligned predicted probabilities with actual frequencies, reducing the Brier score for the best model to 0.00040.
- By optimizing the classification threshold to 0.9036, the system achieved a precision of 95.19%, effectively minimizing the administrative burden of false alarms while maintaining a strong detection rate of over 80%.

In summary, a successful fraud detection system relies on a combination of gradient scaling, rigorous threshold optimization, and probability calibration. The results obtained in this study provide a high-performance, calibrated framework suitable for real-world financial risk management.

Repository and Reproducibility

GitHub Link: <https://github.com/behicekadioglu/CENG463---Take-Home-Midterm/tree/main>

References

- [1] Prof. Dr. Aytuğ Onan, *CENG 463 Machine Learning Course Take Home Midterm*, Izmir Institute of Technology, 2026.
- [2] Machine Learning Group - ULB. (2018). *Credit Card Fraud Detection Dataset*. Kaggle. Available at: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud> (Accessed: 27 April 2026).